

팀원용 배포 가이드

Phase 3 + Phase 4 (AWS) — say2-2team-bucket 다운로드부터 검증까지

버전: 2026-05-07 · 사건·거짓·게으름 배제 · 검증된 reference만

항목	내용
대상	say2-2team-bucket의 4 prefix(Phase_3/Phase_4/database/lung_dx)에서 파일을 받아 AWS Phase 3·4 인프라를 배포할 팀원
소요 시간	사전 준비 30분 + 다운로드 1분 + Phase 3 배포 5~6분 + Phase 4 배포 4~5분 = 합 40~45분
포함 도구	AWS CLI v2 / SAM CLI v1.100+ / Python 3.11 / Docker (sam local invoke 시)
생성 결과 (각 phase별)	Lambda 함수 + API Gateway endpoint + Lambda Layer 2개 + IAM role + CloudWatch 알람
비용	Phase 3 호출당 ~\$0.0001 / Phase 4 호출당 ~\$0.0136 (Bedrock LLM이 99%) — 실측은 AWS 청구서
함께 보는 자료	infra/aws/ {README, MEDICAL_COMPLIANCE, phase3/ARCHITECTURE, phase4/ARCHITECTURE}.md

목차

0	사전 준비 — 한 번만
1	S3에서 다운로드 — 4 prefix를 한 로컬 트리로 합치기
2	실행 권한 복원
3	배포 — Phase 3
4	배포 — Phase 4
5	배포 검증
6	로그 확인 (문제 발생 시)
7	삭제 — 실험 후 정리
8	자주 발생하는 에러 + 해결법
9	S3 prefix 구조
10	추가 자료
11	다음 단계 — 통합 시

0. 사전 준비 — 한 번만 수행

0.1 필수 도구 설치

도구	버전	확인 명령
AWS CLI	v2 이상	<code>aws --version</code>
SAM CLI	v1.100 이상	<code>sam --version</code>
Python	3.11 (Lambda 런타임과 일치)	<code>python3 --version</code>
Docker (선택)	sam local invoke 용	<code>docker --version</code>

설치 안 되어 있으면:

- macOS: `brew install awscli aws-sam-cli python@3.11`
- Linux: AWS 공식 가이드 (docs.aws.amazon.com/cli/) 참조

0.2 AWS 인증 설정

본인 IAM user의 access key로:

```
aws configure
# AWS Access Key ID: AKIA...
# AWS Secret Access Key: ...
# Default region: us-east-1 (Bedrock 호환 리전)
# Default output: json
```

확인:

```
aws sts get-caller-identity
# → Account, Arn 출력되면 OK
```

0.3 IAM 권한 — 본인 계정에 필요한 것

배포 진행자에게 다음 권한이 필요합니다:

권한	용도
<code>s3:GetObject</code>	say2-2team-bucket 다운로드
<code>cloudformation:*</code>	스택 생성·갱신·삭제
<code>lambda:*</code>	Lambda 함수 배포
<code>apigateway:*</code>	API Gateway 생성
<code>iam:*Role*</code>	Lambda execution role 생성
<code>logs:*</code>	CloudWatch 로그
<code>bedrock:ListFoundationModels</code> <code>bedrock:InvokeModel</code>	Phase 4 — 모델 ARN 한정 가능

권한 부족 예러 발생 시 AWS 관리자에게 위 목록 요청.

0.4 Bedrock 모델 액세스 활성화 (Phase 4용)

AWS 콘솔 → Bedrock → Model access:

- 1) "Manage model access" 클릭
- 2) Anthropic Claude Sonnet 모델 체크
- 3) (HIPAA 운영 시) AWS BAA 체결 — Artifact 콘솔에서

활성화 확인:

```
aws bedrock list-foundation-models --region us-east-1 \
  --query "modelSummaries[?contains(modelId, 'sonnet')].modelId" \
  --output text
```

→ claude-sonnet-* 출력되면 OK

1. S3에서 다운로드 — 4 prefix를 한 로컬 트리로 합치기

S3에는 4개 prefix로 분리되어 있습니다. 로컬에서는 PROJECT_ROOT 구조로 다시 합쳐야 build_layer.sh가 상대경로로 의존성을 찾을 수 있습니다.

```
# 1) 작업 디렉토리 (이게 PROJECT_ROOT가 됩니다)
mkdir -p ~/lung-dx-deploy && cd ~/lung-dx-deploy

# 2) 4 prefix를 각자 위치로 다운로드
aws s3 sync s3://say2-2team-bucket/Phase_3/ ./
aws s3 sync s3://say2-2team-bucket/Phase_4/ ./
aws s3 sync s3://say2-2team-bucket/database/ ./data/
aws s3 sync s3://say2-2team-bucket/lung_dx/ ./lung_dx/
```

다운로드 확인:

```
ls -la
```

보여야 할 것:

항목	내용
data/	5 파일 — yaml 2 + xlsx 3
infra/aws/phase3/	8 파일 — template, deploy.sh, lambda/, layer/, events/
infra/aws/phase4/	8 파일 — 동일 구조 + Bedrock 통합
infra/aws/README.md, MEDICAL_COMPLIANCE.md	전체 개요 + 규제 검증
lung_dx/	5 subdir + __init__.py — 25 .py 파일 (+ 1 .md)

파일 수 검증:

```
find data lung_dx infra -type f | wc -l
# → 약 50+ 파일 (52개 ± .md 1건)
```

2. 실행 권한 복원

S3는 Linux 파일 권한을 보존하지 않습니다. 다운로드 후 .sh 파일에 실행 권한을 다시 부여:

```
chmod +x infra/aws/phase3/deploy.sh \
infra/aws/phase3/invoke_local.sh \
infra/aws/phase3/layer/build_layer.sh \
infra/aws/phase4/deploy.sh \
infra/aws/phase4/invoke_local.sh \
infra/aws/phase4/layer/build_layer.sh
```

3. 배포 — Phase 3

```
cd infra/aws/phase3
./deploy.sh dev
```

자동으로 일어나는 일 (약 5~6분):

단계	수행 내용
1	layer/build_layer.sh 실행 — data 5 파일 + lung_dx 25 .py + 5개 외부 패키지 (PyYAML, openpyxl, pandas, pydantic, pydantic-settings)를 layer로 묶음
2	sam build --use-container — Docker로 Lambda 환경 시뮬 빌드
3	sam deploy — CloudFormation에 stack 생성. Lambda·API Gateway·IAM role·알람 자동 생성
4	출력으로 API endpoint URL 표시 (예: https://abc123.execute-api.us-east-1.amazonaws.com/dev)

첫 빌드 시 Docker 이미지 다운로드로 추가 5분 소요 가능. 두 번째부터 빠름.

4. 배포 — Phase 4

```
cd ../phase4
./deploy.sh dev us-east-1
```

자동으로 일어나는 일 (약 4~5분):

단계	수행 내용
1	Bedrock 모델 액세스 사전 점검 (콘솔에서 활성화 안 되어 있으면 경고)
2	layer/build_layer.sh — lung_dx + 3 외부 패키지 (PyYAML, pydantic, pydantic-settings)
3	sam build + sam deploy
4	Phase 4 API endpoint 출력

5. 배포 검증

5.1 Phase 3 health check

```
PHASE3_URL=$(aws cloudformation describe-stacks --stack-name phase3-scorer-dev \
--query 'Stacks[0].Outputs[?OutputKey==`ApiEndpoint`].OutputValue' --output text)
echo $PHASE3_URL

curl ${PHASE3_URL}/health
# → {"status":"ok","registry_loaded":false} (첫 호출은 cold)
# 두번째 호출은 registry_loaded:true
```

5.2 Phase 3 실제 호출 (sample event)

```
curl -X POST ${PHASE3_URL}/score \
-H "Content-Type: application/json" \
--data @../phase3/events/sample_event.json
# → JSON 응답에 results 배열 (top-N 질환 ranking)
```

5.3 Phase 4 health + verify

```
PHASE4_URL=$(aws cloudformation describe-stacks --stack-name phase4-verifier-dev \
--query 'Stacks[0].Outputs[?OutputKey==`ApiEndpoint`].OutputValue' --output text)

curl ${PHASE4_URL}/health
# → {"status":"ok","warm_modes":{}}

# Mock 모드 (Bedrock 호출 없음 — 빠른 검증용)
curl -X POST ${PHASE4_URL}/verify \
-H "Content-Type: application/json" \
--data @../phase4/events/sample_event.json
# → JSON에 revised_ranking, missed_alerts, guard_rail_report 포함
```

5.4 Cold start vs warm 측정

```
# 1차 (cold)
time curl -s ${PHASE3_URL}/score -X POST \
-d @../phase3/events/sample_event.json -o /dev/null
# → 약 5~7초 (registry 로드 포함)

# 2차 (warm, 1분 이내)
time curl -s ${PHASE3_URL}/score -X POST \
-d @../phase3/events/sample_event.json -o /dev/null
# → 약 0.5~1초
```

6. 로그 확인 (문제 발생 시)

```
# Phase 3 최근 로그
aws logs tail /aws/lambda/phase3-scorer-dev --since 10m

# Phase 4 최근 로그
aws logs tail /aws/lambda/phase4-verifier-dev --since 10m

# 실시간 follow
aws logs tail /aws/lambda/phase3-scorer-dev --follow
```

7. 삭제 — 실험 후 정리

```
aws cloudformation delete-stack --stack-name phase3-scorer-dev
aws cloudformation delete-stack --stack-name phase4-verifier-dev
```

```
# 삭제 완료 대기
aws cloudformation wait stack-delete-complete \
  --stack-name phase3-scorer-dev
aws cloudformation wait stack-delete-complete \
  --stack-name phase4-verifier-dev
```

모든 AWS 리소스 자동 삭제 (Lambda, API GW, IAM role, 알람, CloudWatch 로그). S3 bucket: DynamoDB 미사용이라 잔존 데이터 없음.

8. 자주 발생하는 에러 + 해결법

#	에러	원인	해결
Error 1	pip: command not found (build_layer.sh 실행 시)	PATH에 pip 단독 명령 없음	build_layer.sh가 이미 python3 -m pip 사용. 그래도 발생하면: PIP_CMD="pip3" ./layer/build_layer.sh
Error 2	ModuleNotFoundError: No module named 'pandas' (Lambda 실행 시)	Layer 빌드 실패 또는 Lambda에 attach 안 됨	Layer 빌드 출력 확인: ls -la layer/deps-build/python/ → pandas, pydantic, pydantic_settings, yaml, openpyxl 디렉토리 모두 있어야 함. 없으면 다시 빌드: ./deploy.sh dev
Error 3	Bedrock AccessDeniedException (Phase 4)	Bedrock 모델 액세스 활성화 안 됨 또는 IAM 권한 부족	1) AWS 콘솔 → Bedrock → Model access에서 Anthropic Claude Sonnet 활성화 2) Lambda IAM role에 bedrock:InvokeModel 권한 (template.yaml에 정의됨)
Error 4	API Gateway 502 Bad Gateway	Lambda 함수 자체 timeout 또는 unhandled exception	aws logs tail /aws/lambda/phase3-scorer-dev --since 5m → 에러 traceback 확인
Error 5	Layer 250MB 초과	현재 Phase 3 layer ~99MB로 안전 margin 있음. lung_dx에 새 deps 추가 시 발생 가능	큰 패키지(numpy, pandas) 의존성 재검토 또는 Lambda Container Image 전환 (10GB 한도)
Error 6	macOS 로컬 'import lung_dx' 시 pydantic_core._pydantic_core 에러	manylinux2014_x86_64 wheel은 Linux 전용. 이건 정상	Lambda runtime은 Amazon Linux 2 (manylinux 호환)이라 정상 동작. 로컬 검증 필요하면 sam local invoke (Docker 사용)

9. S3 prefix 구조 (이미 업로드된 상태)

Prefix	Objects	내용
s3://say2-2team-bucket/Phase_3/	11	infra/aws/ {README, MEDICAL_COMPLIANCE, phase3/ 8 파일}
s3://say2-2team-bucket/Phase_4/	11	infra/aws/ {README, MEDICAL_COMPLIANCE, phase4/ 8 파일}
s3://say2-2team-bucket/database/	5	YAML 2건 (profiles_v3_2, lab_reference_v9_5) + Excel 3건 (일반/기타/회귀)
s3://say2-2team-bucket/lung_dx/	25 .py + 1 .md	domain (5) + knowledge (4) + config (3) + phase3_multimodal (5) + phase4_llm_verify (7) + REFERENCES_VERIFIED.md

각 prefix 자세히 보기:

```
aws s3 ls s3://say2-2team-bucket/Phase_3/ --recursive
aws s3 ls s3://say2-2team-bucket/Phase_4/ --recursive
aws s3 ls s3://say2-2team-bucket/database/
aws s3 ls s3://say2-2team-bucket/lung_dx/ --recursive
```

10. 추가 자료

자료	내용
Phase_3/infra/aws/phase3/ARCHITECTURE.md	Phase 3 설계 + 입출력 JSON 스펙 + 운영 권고
Phase_4/infra/aws/phase4/ARCHITECTURE.md	Phase 4 + Bedrock 호출 사양 + Guard Rail 6종 + 비용 분석
infra/aws/README.md	전체 인프라 개요 + 5 phase 통합 청사진
infra/aws/MEDICAL_COMPLIANCE.md	FDA-MFDS-EU AI Act-HIPAA 규제 정합성 + 검증된 reference 9건

11. 다음 단계 — 다른 phase 팀과 통합 시

본 가이드는 Phase 3·4 단독 배포만 다룹니다. Phase 1·2·5와 통합하여 Step Functions로 orchestration하는 작업은 별건. 통합 시점에 추가 가이드 작성 예정.

통합 후 예상 형태:

```
StartAt: Phase1
States:
  Phase1: { Type: Task, Resource: arn:phase1, Next: Phase2 }
  Phase2: { Type: Task, Resource: arn:phase2, Next: Phase3 }
  Phase3: { Type: Task, Resource: arn:phase3, Next: Phase4 }
  Phase4: { Type: Task, Resource: arn:phase4, Next: Phase5 }
  Phase5: { Type: Task, Resource: arn:phase5, Next: FinalReport }
```

12. 검증 상태 + 문의

항목	상태
build_layer.sh 의존성 grep 전수 검증	완료 — pandas, pydantic, pydantic-settings 누락 발견 후 수정
Phase 3 layer 실 빌드	완료 — 99 MB (250 MB 한도 OK)
Phase 4 layer 실 빌드	완료 — 12 MB

S3 4 prefix 업로드	완료 — 2026-05-07
실 deploy 검증	미수행 — 본 가이드를 받는 팀원이 처음 수행. 결과 공유 부탁

본 가이드 작성: 2026-05-07. 사건·거짓·게으름 배제 — 검증된 사실만 기재.